



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

**DataQuest: Sistema Web de Validación de Normalización de Bases de
Datos Relacionales**

Curso: BASE DE DATOS II

Docente: Patrick Jose Cuadros Quispe

Integrantes:

Dongo Palza, Manuel Andree	(2023076842)
Perez Peralta, Fabrizio Salvador Elias	(2023077476)

Tacna – Perú
2026



CONTROL DE VERSIONES

Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	EDQ	No especificado	Docente	23/06/2026	Versión Original

DataQuest: Sistema Web de Validación de Normalización de Bases de Datos Relacionales

Informe Visión de Proyecto

Versión 1.1



CONTROL DE VERSIONES

Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	EDQ	No especificado	Docente	23/06/2026	Versión Original

INDICE GENERAL

1. Introducción	5
1.1 Propósito	5
1.2 Alcance	5
1.3 Definiciones, Siglas y Abreviaturas	6
1.4 Referencias.....	6
1.5 Visión General	7
2. Posicionamiento.....	7
2.1 Oportunidad de negocio.....	7
2.2 Definición del problema	7
3. Descripción de los interesados y usuarios	8
3.1 Resumen de los interesados.....	8
3.2 Resumen de los usuarios	9
3.3 Entorno de usuario	9
3.4 Perfiles de los interesados.....	10
3.5 Perfiles de los Usuarios	11
3.6 Necesidades de los interesados y usuarios.....	12
4. Vista General del Producto.....	13
4.1 Perspectiva del producto.....	13
4.2 Resumen de capacidades	14
4.3 Suposiciones y dependencias	15
4.4 Costos y precios.....	16
4.5 Licenciamiento e instalación	16
5. Características del producto.....	17
6. Restricciones	19
7. Rangos de calidad.....	20



8. Precedencia y Prioridad.....	20
9. Otros requerimientos del producto	22
a) Estándares aplicables	22
b) Estándares legales.....	22
c) Estándares de comunicación	22
d) Estándares de cumplimiento de la plataforma	22
e) Estándares de calidad y seguridad	22
f) Estándares de datos y trazabilidad	22
CONCLUSIONES	23
RECOMENDACIONES	23
BIBLIOGRAFÍA.....	24



1. Introducción

1.1 Propósito

El propósito de este documento es establecer una visión clara, técnica y compartida del sistema DataQuest, una plataforma web orientada a la validación de normalización de bases de datos relacionales. El documento describe el problema que atiende el proyecto, los usuarios involucrados, las capacidades esperadas, los requerimientos funcionales, las restricciones, los rangos de calidad y los criterios de prioridad que deben guiar el desarrollo y la evaluación del producto.

A diferencia de una descripción general, este FD02 se elabora a partir de la revisión directa del ZIP del sistema. Se analizaron archivos de entrada como README.md, app.py, api.py, controladores, módulos core, conexión a base de datos, scripts SQL, Dockerfile, Terraform, workflows de GitHub Actions, pruebas y documentación de integración API. Por tanto, el contenido refleja tanto la intención funcional del proyecto como el estado técnico observado en los archivos revisados.

El documento servirá como línea base para alinear al equipo de desarrollo, docente, evaluadores y usuarios objetivo sobre qué debe lograr DataQuest, qué módulos son prioritarios y qué aspectos deben corregirse antes de una entrega final o despliegue productivo.

1.2 Alcance

DataQuest estará compuesto por una aplicación web desarrollada principalmente en Python, con interfaz Streamlit, motor de diagnóstico de normalización, conexión a Supabase/PostgreSQL, módulo de autenticación, dashboard de actividad, panel de comunidad, historial de validaciones, visualización de esquemas y un servicio API construido con FastAPI para consumo externo del motor de normalización.

El alcance funcional incluye la recepción de esquemas de bases de datos mediante SQL pegado o archivos compatibles, extracción de tablas, columnas, claves primarias, claves foráneas y dependencias funcionales, evaluación de cumplimiento de 1FN, 2FN y 3FN, generación de sugerencias de corrección, producción de un SQL normalizado referencial, visualización de diagramas ER mediante Mermaid y registro del historial de análisis por usuario.

El alcance técnico incluye despliegue containerizado con Docker, variables de entorno para Supabase, publicación en Azure App Service mediante Terraform y automatización CI/CD con GitHub Actions, pruebas, análisis de seguridad y construcción de imagen Docker. El alcance no incluye validación matemática completa de todas las formas normales avanzadas ni reemplaza la revisión profesional de un diseñador de base de datos. El archivo core/dependencias.py conserva funciones de clausura y claves candidatas marcadas como TODO, por lo que el análisis actual se apoya principalmente en reglas y heurísticas implementadas.

Asimismo, se identifica una inconsistencia técnica importante: README.md y db/modelos.sql plantean el uso de Supabase Auth y auth.users, mientras que db/auth.py implementa un login manual basado en una tabla usuarios con hash SHA-256 y tokens dummy. Esta diferencia deberá resolverse para consolidar el alcance real de autenticación antes de considerar el producto como cerrado.



1.3 Definiciones, Siglas y Abreviaturas

Sigla / Término	Definición
API	Interfaz de Programación de Aplicaciones que permite consumir servicios desde otros sistemas. En DataQuest se expone mediante FastAPI.
BD	Base de Datos. Conjunto estructurado de datos organizado mediante tablas, relaciones, restricciones y reglas de integridad.
CI/CD	Integración y Despliegue Continuo. En el proyecto se observa mediante workflows de GitHub Actions para pruebas, seguridad y despliegue.
Docker	Tecnología de contenedores utilizada para empaquetar la aplicación Streamlit y facilitar su despliegue.
FastAPI	Framework Python usado para exponer el endpoint /api/normalizacion.
FN	Forma Normal. Regla de diseño relacional que busca reducir redundancia y anomalías.
1FN	Primera Forma Normal. Exige atributos atómicos, ausencia de grupos repetitivos y existencia de identificación de registros.
2FN	Segunda Forma Normal. Exige cumplimiento de 1FN y ausencia de dependencias parciales respecto a claves primarias compuestas.
3FN	Tercera Forma Normal. Exige cumplimiento de 2FN y ausencia de dependencias transitivas entre atributos no clave.
Mermaid	Lenguaje de diagramación usado para representar esquemas relacionales y relaciones detectadas.
MVP	Producto Mínimo Viable. Versión inicial del sistema con funcionalidades esenciales para validar su utilidad.
PostgreSQL	Sistema gestor de base de datos relacional utilizado a través de Supabase.
RLS	Row Level Security. Políticas de seguridad en PostgreSQL/Supabase para restringir acceso por usuario.
Streamlit	Framework Python empleado para construir la interfaz web interactiva de DataQuest.
Supabase	Backend as a Service que provee base de datos PostgreSQL, autenticación, API y funciones Realtime.
Terraform	Herramienta de infraestructura como código utilizada para definir recursos Azure.

1.4 Referencias

- README.md del proyecto DataQuest / Validador de Normalización de Bases de Datos Relacionales.
- INTEGRACION_API.md del proyecto, donde se documenta el endpoint público de normalización.
- db/modelos.sql, script de base de datos para Supabase/PostgreSQL con perfiles, presencia, historial y políticas RLS.



- Archivos app.py, api.py, controllers/, core/, db/, views/, visualizacion/, Dockerfile, requirements.txt, Terraform y workflows CI/CD revisados en el ZIP.
- Plantilla FD02-EPIS Informe Visión de Proyecto utilizada como estructura base del documento.

1.5 Visión General

Este documento presenta la visión del sistema DataQuest desde una perspectiva funcional, técnica y académica. Primero se define la oportunidad de negocio y el problema que se busca resolver. Luego se describen los interesados, usuarios, entorno de uso y necesidades principales. Posteriormente se desarrolla la vista general del producto, sus capacidades, dependencias, costos, instalación y licenciamiento. Finalmente se enumeran los requerimientos funcionales, restricciones, rangos de calidad, prioridades y otros estándares necesarios para su desarrollo.

La visión central del proyecto es convertir DataQuest en una herramienta educativa y técnica que permita a estudiantes, docentes, desarrolladores y analistas de datos diagnosticar esquemas relacionales, entender por qué no cumplen determinadas formas normales y obtener recomendaciones concretas para mejorar su diseño sin depender de inteligencia artificial.

2. Posicionamiento

2.1 Oportunidad de negocio

En cursos de bases de datos, proyectos académicos y entornos de desarrollo, la normalización de esquemas relacionales suele verificarse manualmente mediante teoría, revisión del docente, diagramas hechos a mano o herramientas que no explican claramente el motivo de una violación. Esto genera una brecha entre la teoría de 1FN, 2FN y 3FN y su aplicación práctica sobre sentencias SQL reales.

DataQuest representa una oportunidad para transformar esta revisión en un proceso guiado, automatizado y visual. El sistema permite pegar o cargar un esquema, analizar tablas, detectar problemas de normalización, presentar hallazgos y generar una versión corregida referencial. Además, al contar con historial por usuario, dashboard, comunidad y API, puede ser usado tanto como herramienta académica en laboratorio como servicio integrable para otros proyectos.

Desde el punto de vista del producto, DataQuest puede diferenciarse porque no se limita a decir si un esquema está mal, sino que explica el nivel alcanzado, clasifica las violaciones, propone acciones con impacto y confianza, genera un SQL sugerido y muestra una vista ER actualizada. Esta combinación aporta valor educativo, operativo y técnico.

2.2 Definición del problema

Los usuarios que diseñan bases de datos relacionales enfrentan dificultad para validar si sus esquemas cumplen correctamente las formas normales, especialmente cuando existen tablas con dependencias parciales, transitivas, grupos repetitivos o ausencia de claves primarias. La revisión manual consume tiempo, depende del criterio del evaluador y puede generar resultados poco consistentes.



Elemento	Descripción
El problema de	Validación manual, dispersa y poco trazable del nivel de normalización de esquemas de bases de datos relacionales.
Afecta a	Estudiantes de bases de datos, docentes, desarrolladores, analistas de datos, administradores de bases de datos y equipos que diseñan esquemas relacionales.
Cuyo impacto es	Errores de diseño, redundancia, anomalías de inserción/actualización/borrado, pérdida de tiempo en revisión, baja comprensión de dependencias funcionales y dificultad para justificar cambios estructurales.
Una solución exitosa	Permitiría cargar o pegar SQL, extraer tablas y restricciones, diagnosticar 1FN/2FN/3FN, mostrar violaciones, sugerir correcciones, generar SQL referencial, visualizar diagramas, guardar historial y exponer una API de diagnóstico para integración externa.

3. Descripción de los interesados y usuarios

3.1 Resumen de los interesados

Nombre	Representa	Rol
Docente / Sponsor Académico	Evaluación del proyecto y validación del cumplimiento de los entregables FD01/FD02.	Sponsor, evaluador y aprobador académico.
Equipo de Desarrollo DataQuest	Estudiantes responsables del diseño, implementación, pruebas, documentación y despliegue.	Proveedor técnico de la solución.
Usuario Académico / Estudiante	Usuarios que necesitan validar esquemas de bases de datos durante prácticas o trabajos.	Usuario final principal y fuente de retroalimentación.
Administrador del Sistema	Responsable de configuración, usuarios, Supabase, variables de entorno, despliegue y seguridad.	Operador técnico y mantenedor de plataforma.
Docente de Base de Datos / Validador Funcional	Especialista que revisa la lógica de normalización y la coherencia de las sugerencias.	Validador del dominio técnico-relacional.



Nombre	Representa	Rol
Equipo Externo Consumidor de API	Desarrolladores que integran el endpoint de normalización en otros productos.	Consumidor técnico de la API REST.

3.2 Resumen de los usuarios

Nombre	Rol	Descripción
Usuario Estudiante	Usuario funcional académico	Carga esquemas, revisa diagnóstico, consulta sugerencias y descarga SQL normalizado para aprender o corregir su trabajo.
Usuario Docente / Evaluador	Usuario revisor	Utiliza el sistema para verificar ejercicios, comparar niveles de normalización y revisar el historial de validaciones.
Usuario Desarrollador	Usuario técnico	Valida esquemas reales, usa la API de normalización y revisa salidas JSON para integrarlas en otros sistemas.
Usuario Administrador	Responsable técnico	Gestiona configuración, acceso, base de datos, despliegue, seguridad, monitoreo y mantenimiento del sistema.

3.3 Entorno de usuario

El sistema opera como aplicación web responsiva construida con Streamlit. Los usuarios acceden desde navegadores modernos como Chrome, Edge o Firefox, sin instalación local del cliente. Para el usuario final, el flujo se basa en autenticación, acceso al dashboard, ingreso al validador, carga o pegado de SQL, revisión de resultados, aplicación de sugerencias y consulta del historial.

En ambiente local, el proyecto se ejecuta con `streamlit run app.py` en el puerto 8501. El servicio API se ejecuta con `uvicorn api:app` y expone `/api/normalizacion`. En despliegue productivo, los archivos Terraform definen dos Azure Linux Web Apps: una para Streamlit en el puerto 8501 y otra para FastAPI en el puerto 8000. Las variables `SUPABASE_URL`, `SUPABASE_KEY` y `DATABASE_URL` son requeridas para conectarse a Supabase/PostgreSQL.

La carga esperada para una versión académica es moderada: usuarios conectados simultáneamente, validaciones por SQL pegado, consultas al historial y actualización de presencia. Para uso institucional o público, se deberá reforzar autenticación, límites de tamaño de SQL, manejo de concurrencia, protección de claves, control de CORS y monitoreo de consumo de recursos.



3.4 Perfiles de los interesados

Docente / Sponsor Académico

Representante	Docente responsable del curso o evaluador del proyecto.
Tipo	Interesado principal y sponsor académico.
Responsabilidades	Revisar el alcance, evaluar el documento de visión, validar que el producto responda a objetivos académicos y aprobar entregables.
Grado de participación	Alto durante revisión, sustentación y validación final.
Comentarios	Requiere trazabilidad entre el sistema revisado, los requerimientos y el informe presentado.

Equipo de Desarrollo DataQuest

Representante	Equipo responsable del proyecto.
Tipo	Proveedor técnico y responsable de construcción.
Responsabilidades	Implementar interfaz Streamlit, API FastAPI, motor de normalización, base de datos, seguridad, pruebas, despliegue y documentación.
Grado de participación	Crítico durante todo el ciclo de desarrollo.
Comentarios	Debe corregir inconsistencias detectadas entre documentación, base de datos y código de autenticación.

Validador Funcional de Bases de Datos

Representante	Docente o especialista en diseño relacional.
Tipo	Experto de dominio.
Responsabilidades	Validar reglas de normalización, dependencias funcionales, sugerencias de corrección y generación SQL.
Grado de participación	Alto en validación de core/diagnostico.py, validadores 1FN/2FN/3FN y generador_sql.py.



Comentarios	Debe diferenciar entre heurísticas útiles y validación formal completa.
--------------------	---

Administrador Técnico

Representante	Responsable de infraestructura, Supabase y despliegue.
Tipo	Interesado técnico-operativo.
Responsabilidades	Configurar variables de entorno, base de datos, políticas RLS, Azure App Service, Docker, seguridad, backups y monitoreo.
Grado de participación	Medio a alto durante puesta en marcha y mantenimiento.
Comentarios	Requiere documentación clara de instalación, dependencias, secretos y actualización de modelos.sql.

3.5 Perfiles de los Usuarios

Usuario Estudiante

Representante	Estudiante que diseña bases de datos relacionales.
Rol	Usuario funcional principal.
Responsabilidades	Ingresar SQL, revisar diagnóstico, seleccionar nivel objetivo, aplicar sugerencias, descargar script y consultar historial.
Nivel de experiencia	Básico a medio en SQL y teoría de normalización.
Necesidades principales	Interfaz simple, mensajes claros, explicación de violaciones, ejemplos y generación de salida comprensible.

Usuario Docente / Evaluador

Representante	Docente o asistente de curso.
Rol	Revisor académico.
Responsabilidades	Verificar si el sistema ayuda a evaluar diseños de bases de datos y si sus sugerencias son coherentes.



Nivel de experiencia	Medio a avanzado en modelado relacional y normalización.
Necesidades principales	Resultados justificables, trazabilidad de validaciones, claridad de reglas y exportación de evidencias.

Usuario Desarrollador / Integrador API

Representante	Desarrollador que consume el endpoint público.
Rol	Consumidor técnico del servicio.
Responsabilidades	Enviar JSON con SQL al endpoint, procesar respuesta, manejar errores e integrar diagnóstico en otro sistema.
Nivel de experiencia	Medio a avanzado en programación web y APIs REST.
Necesidades principales	API estable, CORS documentado, estructura JSON clara, códigos de error, documentación y límites de uso.

Usuario Administrador

Representante	Responsable de operación técnica de DataQuest.
Rol	Administrador del sistema.
Responsabilidades	Gestionar cuentas, conexión Supabase, presencia, historial, variables de entorno, despliegue y seguridad.
Nivel de experiencia	Medio a avanzado en sistemas web, cloud y base de datos.
Necesidades principales	Panel de control, logs, configuración segura, backups, monitoreo y documentación de incidentes.

3.6 Necesidades de los interesados y usuarios

Necesidad	Prioridad	Preocupaciones	Solución Propuesta
Validar normalización de esquemas SQL	Crítica	Que la revisión manual sea lenta, subjetiva o incompleta.	Parser SQL y motor de diagnóstico 1FN, 2FN y 3FN con reporte estructurado.



Necesidad	Prioridad	Preocupaciones	Solución Propuesta
Identificar violaciones específicas	Crítica	Que el usuario no entienda por qué su esquema no cumple una forma normal.	Listas de violaciones por nivel con mensajes explicativos y SQL afectado.
Recibir sugerencias accionables	Crítica	Que el sistema solo muestre errores sin orientar la corrección.	Módulo corrector con acciones, impacto, confianza, beneficios y tipo de mejora.
Generar un script normalizado referencial	Alta	Que el usuario deba reconstruir manualmente el nuevo esquema.	Generador SQL que aplica sugerencias seleccionadas y produce CREATE TABLE referencial.
Visualizar el esquema y relaciones	Alta	Que el resultado sea difícil de comprender solo en texto.	Generación de diagrama ER Mermaid y vista previa en la página de resultados.
Guardar historial por usuario	Alta	Perder evidencias de análisis anteriores o no poder revisar avances.	Tabla historial_validaciones con JSONB, fecha, niveles, sugerencias y RLS por usuario.
Acceder con cuenta y sesión	Alta	Que usuarios no autorizados accedan a datos o historial.	Autenticación y control de sesión; debe alinearse entre Supabase Auth y login manual actual.
Ver comunidad activa	Media	No contar con interacción o presencia de usuarios conectados.	Tabla presencia, actualización de actividad y página Comunidad.
Consumir el motor desde otros sistemas	Media	Que la herramienta solo funcione desde la interfaz Streamlit.	API FastAPI POST /api/normalizacion con respuesta JSON.
Desplegar y mantener el sistema	Alta	Que el sistema solo funcione localmente o no tenga camino de publicación.	Dockerfile, Terraform, GitHub Actions, Azure App Service y variables de entorno.

4. Vista General del Producto

4.1 Perspectiva del producto

DataQuest es un sistema de información web orientado al análisis de esquemas relacionales. Funciona como plataforma independiente para usuarios humanos mediante Streamlit y como servicio consumible por otros sistemas mediante FastAPI. Su arquitectura se organiza bajo una adaptación del patrón Modelo-Vista-Controlador: vistas en views/ y app.py, controladores en controllers/, lógica de negocio en core/ y acceso a datos en db/.

La plataforma no pretende sustituir la enseñanza formal de normalización ni garantizar una validación matemática completa para todos los casos posibles. Su valor se encuentra en



automatizar reglas frecuentes, detectar patrones problemáticos, explicar violaciones y apoyar la corrección estructural de esquemas SQL. El análisis se realiza mediante algoritmos y heurísticas, no mediante inteligencia artificial.

El producto se apoya en Supabase/PostgreSQL para persistencia de perfiles, presencia e historial. Las políticas RLS definidas en modelos.sql buscan que cada usuario acceda únicamente a su propio perfil e historial, mientras la presencia puede ser visible a todos. La solución también contempla despliegue en nube mediante Azure Linux Web Apps, imagen Docker y automatización con GitHub Actions.

Capa	Archivos / Tecnología	Responsabilidad observada
Vista	app.py, views/00_auth.py, 00_inicio.py, 03_comunidad.py, 04_validador.py, 05_resultados.py, 06_historial.py	Renderiza interfaz, navegación, autenticación visual, dashboard, validador, resultados, comunidad e historial.
Controlador	controllers/auth_controller.py, comunidad_controller.py, dashboard_controller.py, validacion_controller.py	Orquesta sesión, presencia, métricas, parsing, diagnóstico, sugerencias y persistencia de historial.
Core / Modelo lógico	core/parser.py, diagnostico.py, validador_1fn.py, validador_2fn.py, validador_3fn.py, corrector.py, generador_sql.py	Extrae esquemas SQL, valida formas normales, genera sugerencias y aplica mejoras referenciales.
Datos	db/conexion.py, db/auth.py, db/realtime.py, db/modelos.sql	Conecta Supabase/PostgreSQL, gestiona autenticación/presencia e historial.
API	api.py, INTEGRACION_API.md	Expone diagnóstico de normalización mediante POST /api/normalizacion.
Despliegue	Dockerfile, terraform/, .github/workflows/	Conteneriza, despliega en Azure y ejecuta CI/CD con pruebas y seguridad.

4.2 Resumen de capacidades

Capacidad	Descripción Breve
Autenticación y sesión	Registro, login, sesión en Streamlit, perfil de usuario y cierre de sesión. Se debe unificar el enfoque de Supabase Auth versus tabla usuarios manual.
Dashboard de actividad	Métricas de validaciones realizadas, tablas analizadas, reportes/hallazgos y usuarios online.
Entrada de esquemas	Recepción de SQL pegado o archivo .sql, .csv, .xlsx, .txt desde la vista Validador.



Capacidad	Descripción Breve
Parser SQL	Extracción de tablas, columnas, claves primarias, claves foráneas y SQL original desde sentencias CREATE TABLE.
Dependencias funcionales	Recepción de dependencias funcionales escritas por el usuario e inferencia heurística desde nombres de columnas.
Diagnóstico 1FN	Detección de ausencia de clave primaria y posibles grupos repetitivos o atributos multivaluados.
Diagnóstico 2FN	Identificación de dependencias parciales cuando existe clave primaria compuesta.
Diagnóstico 3FN	Detección de dependencias transitivas entre atributos no clave.
Sugerencias de corrección	Generación de acciones con nivel, impacto, confianza, beneficios, tipo de mejora y detalle.
Aplicación de mejoras	Construcción de un esquema final referencial y SQL generado a partir de sugerencias seleccionadas.
Visualización ER	Generación de Mermaid ER con entidades, atributos PK/FK y relaciones inferidas por FK explícita o patrón _id/id_.
Historial de validaciones	Persistencia de esquema original, final, niveles, violaciones, sugerencias y dependencias en JSONB por usuario.
Comunidad en vivo	Registro de presencia, actualización de última actividad y listado de usuarios conectados.
API pública	Endpoint POST /api/normalizacion que devuelve success, nivel_actual, violaciones, mejoras y resumen en JSON.
DevOps y despliegue	Docker, Azure App Service, Terraform, GitHub Actions, Semgrep, Snyk y tests automatizados.

4.3 Suposiciones y dependencias

- El equipo de desarrollo contará con acceso al repositorio, ZIP actualizado, archivo .env local y credenciales válidas de Supabase para ejecutar las funciones de base de datos.
- La plataforma se ejecutará con Python 3.10 o superior; el Dockerfile usa Python 3.11 slim, por lo que se considera compatible con el stack declarado.
- Supabase/PostgreSQL estará disponible para almacenar perfiles, presencia e historial de validaciones; sin esta conexión, el validador puede analizar SQL, pero no completar funciones de usuario, historial o comunidad.
- La API FastAPI se desplegará de forma separada o con comando específico, ya que el Dockerfile por defecto inicia Streamlit y Terraform define una App adicional para api.py con uvicorn.
- El parser actual está orientado principalmente a sentencias CREATE TABLE. La lectura de CSV se encuentra como función vacía en core/parser.py, por lo que el soporte real para CSV/XLS debe completarse antes de declararlo funcional.
- Las funciones de clausura y claves candidatas en core/dependencias.py están pendientes, por lo que el diagnóstico actual depende de reglas implementadas y heurísticas de nombres.



- Los workflows declaran pruebas BDD en tests/bdd, pero esa carpeta no se observó en la estructura listada. Debe crearse o ajustarse el workflow para evitar fallos en CI.
- La seguridad de autenticación debe reforzarse porque db/auth.py utiliza SHA-256 básico y tokens dummy, mientras el modelo de datos documentado propone Supabase Auth con JWT y RLS.

4.4 Costos y precios

El costo comercial del producto no se encuentra especificado en los archivos revisados. Para mantener coherencia con el alcance académico, DataQuest se considera un proyecto académico/prototipo funcional, sin precio formal de venta definido dentro del ZIP.

Para un escenario de operación real, los costos a estimar deberían incluir hosting en Azure App Service, Supabase, dominio, certificado SSL si no se usa uno administrado, monitoreo, almacenamiento, mantenimiento del motor de validación, soporte técnico, pruebas de seguridad, revisión de dependencias, CI/CD y posibles costos por consumo de API o transferencia.

Concepto	Estado en archivos revisados	Observación
Costo de desarrollo	No especificado	Debe definirse si el proyecto será comercializado o entregado a un tercero.
Hosting web	Parcialmente definido	Terraform define Azure Linux Web App y Service Plan B1, pero no se incluye presupuesto monetario.
Base de datos	Definido técnicamente	Supabase/PostgreSQL requerido; costo depende del plan contratado.
Dominio y SSL	No especificado	Debe definirse para despliegue público.
Mantenimiento	No especificado	Debe cubrir actualización de dependencias, seguridad, parser, reglas y soporte.

4.5 Licenciamiento e instalación

El README indica licencia MIT para el proyecto. Bajo esa premisa, el software podría usarse, modificarse y distribuirse conforme a los términos de dicha licencia. No obstante, en los archivos revisados no se observó el contenido del archivo LICENSE en el listado principal, por lo que se recomienda incorporar o verificar dicho archivo antes de una entrega formal.

La instalación local requiere clonar o descomprimir el proyecto, crear un entorno Python, instalar requirements.txt, configurar las variables SUPABASE_URL, SUPABASE_KEY y DATABASE_URL, ejecutar el script db/modelos.sql en Supabase y levantar la aplicación con streamlit run app.py. Para la API, se debe ejecutar uvicorn api:app --host 0.0.0.0 --port 8000.

El despliegue contenerizado utiliza Dockerfile basado en python:3.11-slim, instalación de dependencias del sistema operativo, instalación de requirements.txt y exposición del puerto



8501. Para Azure, Terraform define recursos `azure_rm_resource_group`, `azure_rm_service_plan`, `azure_rm_linux_web_app` para Streamlit y otra Web App para API.

5. Características del producto

A continuación, se detallan los requerimientos funcionales identificados y ampliados a partir del análisis del ZIP del sistema DataQuest. Se incluyen tanto funcionalidades implementadas como aquellas documentadas pero que requieren consolidación técnica.

ID	Descripción del Requerimiento Funcional
RF-01	El sistema debe permitir crear una cuenta de usuario y asociarla a un perfil público con nombre y fecha de creación.
RF-02	El sistema debe permitir iniciar sesión y mantener el estado activo del usuario en <code>st.session_state</code> durante la navegación.
RF-03	El sistema debe restringir el acceso a Validador, Resultados, Historial y Comunidad cuando no exista sesión activa.
RF-04	El sistema debe registrar o actualizar la presencia del usuario conectado y mostrar la comunidad activa.
RF-05	El sistema debe presentar un dashboard inicial con métricas de validaciones, tablas analizadas, reportes/hallazgos y usuarios online.
RF-06	El sistema debe permitir ingresar esquemas mediante pegado directo de SQL.
RF-07	El sistema debe permitir cargar archivos <code>.sql</code> , <code>.txt</code> , <code>.csv</code> y <code>.xlsx</code> ; para CSV/XLS debe completarse la implementación real de lectura estructurada.
RF-08	El sistema debe parsear sentencias <code>CREATE TABLE</code> , identificando nombre de tabla, columnas, claves primarias, claves foráneas y SQL original.
RF-09	El sistema debe permitir registrar dependencias funcionales explícitas en formato <code>A, B -> C</code> cuando el usuario las proporcione.
RF-10	El sistema debe inferir dependencias funcionales heurísticas cuando no existan dependencias ingresadas por el usuario.



ID	Descripción del Requerimiento Funcional
RF-11	El sistema debe diagnosticar cumplimiento de 1FN detectando ausencia de clave primaria y posibles grupos repetitivos o atributos multivaluados.
RF-12	El sistema debe diagnosticar cumplimiento de 2FN identificando dependencias parciales en tablas con clave primaria compuesta.
RF-13	El sistema debe diagnosticar cumplimiento de 3FN identificando dependencias transitivas entre atributos no clave.
RF-14	El sistema debe mostrar el nivel actual de normalización del esquema: sin normalizar, 1FN, 2FN o 3FN.
RF-15	El sistema debe permitir seleccionar nivel objetivo de normalización: 1FN, 2FN o 3FN.
RF-16	El sistema debe generar sugerencias de corrección con nivel, acción, detalle, impacto, confianza, beneficios y tipo de mejora.
RF-17	El sistema debe permitir seleccionar sugerencias y aplicar mejoras para construir un esquema final referencial.
RF-18	El sistema debe generar y permitir descargar un script SQL resultante en formato .sql o .txt.
RF-19	El sistema debe visualizar una vista previa del esquema mediante código Mermaid ER renderizado en la interfaz.
RF-20	El sistema debe guardar en historial el esquema original, diagnóstico, dependencias, violaciones, nivel inicial y datos de entrada.
RF-21	El sistema debe actualizar el historial con nivel objetivo, nivel final, sugerencias aplicadas y esquema final.
RF-22	El sistema debe permitir consultar el historial de validaciones del usuario autenticado, ordenado por fecha descendente.
RF-23	El sistema debe mostrar el detalle de una validación histórica: SQL original, Mermaid final, errores y soluciones aplicadas.
RF-24	El sistema debe exponer una API REST que reciba JSON con campo sql y devuelva diagnóstico estructurado de normalización.
RF-25	El sistema debe devolver errores controlados cuando el SQL esté vacío, no contenga tablas válidas o ocurra un error interno.
RF-26	El sistema debe contar con configuración de variables de entorno para conexión a Supabase y PostgreSQL.
RF-27	El sistema debe contar con scripts SQL de creación de tablas, triggers, políticas RLS, permisos e integración Realtime.



ID	Descripción del Requerimiento Funcional
RF-28	El sistema debe contar con despliegue contenerizado mediante Docker y configuración de infraestructura mediante Terraform.
RF-29	El sistema debe contar con workflows de pruebas, análisis de seguridad y despliegue automático en GitHub Actions.
RF-30	El sistema debe documentar claramente las funcionalidades implementadas, pendientes y restricciones reales del motor de normalización.

6. Restricciones

Las restricciones del producto delimitan aquello que debe respetarse durante la implementación, despliegue, validación y uso de DataQuest.

- Técnicas: El sistema debe ejecutarse con Python 3.10 o superior; el contenedor oficial utiliza Python 3.11 slim. La interfaz depende de Streamlit y la API depende de FastAPI/Uvicorn.
- Base de datos: El funcionamiento completo depende de Supabase/PostgreSQL y de la correcta ejecución del script db/modelos.sql. Sin embargo, debe añadirse o ajustar la tabla usuarios si se conserva el login manual actual.
- Autenticación: Debe resolverse la inconsistencia entre Supabase Auth documentado y autenticación manual implementada. El uso de SHA-256 básico y tokens dummy no es recomendable para producción.
- Parser: El análisis real se limita principalmente a CREATE TABLE. El soporte para CSV/XLS está documentado, pero parse_csv retorna una lista vacía, por lo que aún no debe considerarse completo.
- Normalización: Las funciones de clausura y claves candidatas se encuentran pendientes. Por ello, las detecciones de dependencias no deben presentarse como prueba matemática exhaustiva.
- API: El endpoint público tiene CORS abierto a todos los orígenes. Para producción debe restringirse o controlarse según dominios, autenticación, cuota y tamaño del request.
- Despliegue: El Dockerfile inicia Streamlit por defecto; la API requiere comando específico uvicorn api:app o una imagen/servicio separado como define Terraform.
- CI/CD: El workflow de pruebas referencia tests/bdd, pero esta carpeta no aparece en los archivos listados. Debe agregarse o corregirse el pipeline.
- Presupuesto: No existe presupuesto comercial confirmado en los archivos revisados. Cualquier precio debe declararse como supuesto externo y no como dato del proyecto.
- Cronograma: No se encontró cronograma detallado confirmado en los archivos revisados; solo se evidencia estado en planificación y roadmap pendiente en README.md.



7. Rangos de calidad

Rango de calidad	Criterio esperado
Seguridad	El sistema debe proteger credenciales, sesiones, claves de Supabase y datos del historial. Para producción debe reemplazarse SHA-256 básico por bcrypt/Argon2 o Supabase Auth real, eliminar tokens dummy, validar RLS y restringir CORS.
Disponibilidad	En entorno académico se espera disponibilidad durante sesiones de uso y sustentación. En despliegue Azure se recomienda objetivo mínimo de 99%, con ventanas de mantenimiento programadas.
Rendimiento	El parseo de SQL y diagnóstico de esquemas pequeños/medianos debe responder en menos de 5 segundos. La API debe manejar errores y evitar bloqueos por SQL excesivamente grande.
Usabilidad	La interfaz debe ser clara para estudiantes, con mensajes comprensibles, pasos guiados, selección de nivel objetivo, botones de descarga y explicación de errores.
Mantenibilidad	El patrón MVC debe conservarse; core, controllers, db, views y visualización deben mantenerse separados para facilitar cambios.
Portabilidad	El sistema debe poder ejecutarse localmente, con Docker y en Azure Linux Web App mediante variables de entorno.
Escalabilidad	Para mayor carga se requiere limitar tamaño de entrada, paginar historial, controlar consultas Supabase y separar servicios web/API.
Confiabilidad	Las sugerencias deben indicar cuando son heurísticas. Deben agregarse pruebas unitarias reales para parser, validadores, generador SQL y API.
Privacidad	Cada usuario debe visualizar solo su historial. Las políticas RLS deben mantenerse activas y probadas.
Observabilidad	Se recomienda añadir logs estructurados, manejo de errores, auditoría de request API y monitoreo de Azure/Supabase.

8. Precedencia y Prioridad

ID	Requerimiento Funcional	Prioridad	Justificación
RF-08	Parser de sentencias CREATE TABLE	Alta	Sin extracción de tablas y columnas no existe diagnóstico.
RF-11	Validación de 1FN	Alta	Es el primer nivel de normalización y base de evaluación.



ID	Requerimiento Funcional	Prioridad	Justificación
RF-12	Validación de 2FN	Alta	Permite detectar dependencias parciales en claves compuestas.
RF-13	Validación de 3FN	Alta	Aporta valor central al identificar dependencias transitivas.
RF-16	Sugerencias de corrección	Alta	Diferencia al sistema de un simple detector de errores.
RF-17	Aplicación de mejoras	Alta	Permite pasar del diagnóstico a una propuesta de solución.
RF-18	Descarga de SQL resultante	Alta	Entrega un resultado reutilizable para el usuario.
RF-01/RF-02/RF-03	Autenticación y protección de páginas	Alta	Protege datos e historial, pero debe corregirse su implementación productiva.
RF-20/RF-21/RF-22	Historial de validaciones	Alta	Permite trazabilidad académica y seguimiento de avances.
RF-24/RF-25	API de normalización	Media	Amplía el valor del sistema hacia integraciones externas.
RF-05	Dashboard de actividad	Media	Aporta seguimiento, pero depende de historial y presencia.
RF-19	Visualización Mermaid ER	Media	Facilita comprensión visual, pero no reemplaza el diagnóstico.
RF-04	Comunidad en vivo	Media	Agrega valor social, pero no es núcleo del validador.
RF-27	Políticas RLS y modelos SQL	Alta	Base de seguridad de perfiles, presencia e historial.
RF-28/RF-29	Docker, Terraform y CI/CD	Media	Importante para despliegue; puede consolidarse después de completar core funcional.
RF-07	CSV/XLS real	Baja	Está documentado, pero no implementado; puede ser fase posterior al MVP.
RF-30	Documentación de pendientes	Alta	Evita sobreprometer capacidades no terminadas.



9. Otros requerimientos del producto

a) Estándares aplicables

El sistema debe mantener buenas prácticas de desarrollo Python, separación de responsabilidades y documentación técnica suficiente para revisión académica. La interfaz debe respetar principios de usabilidad web, claridad visual, navegación consistente y mensajes de error comprensibles. El código debería formatearse con Black, revisarse con Flake8 y probarse con Pytest, según las dependencias declaradas en requirements-dev.txt.

b) Estándares legales

DataQuest almacena datos de usuarios, presencia e historial de validaciones. Por ello debe respetar principios de privacidad, consentimiento, finalidad, minimización y seguridad de datos. Si se publica como servicio abierto, deberá incorporar términos de uso, política de privacidad, tratamiento de datos y advertencia sobre el carácter referencial de los diagnósticos.

c) Estándares de comunicación

La comunicación interna entre vistas, controladores, core y base de datos debe mantenerse mediante funciones claramente definidas. La API pública debe usar HTTP POST, JSON, respuestas estructuradas, códigos de error consistentes, validación del payload, control de tamaño de entrada y documentación OpenAPI disponible en /docs.

d) Estándares de cumplimiento de la plataforma

El sistema debe ser compatible con ejecución local, Docker y Azure Linux Web App. La configuración debe depender de variables de entorno y no de credenciales embebidas. Los recursos Terraform deben parametrizarse correctamente y los secretos deben administrarse mediante GitHub Secrets o el mecanismo seguro del proveedor cloud.

e) Estándares de calidad y seguridad

- Desarrollo: usar control de versiones Git, ramas protegidas, pruebas unitarias reales, integración continua y revisión de cambios antes de producción.
- Seguridad: reemplazar hash SHA-256 básico por un mecanismo robusto, usar Supabase Auth o una tabla usuarios coherente, evitar tokens dummy y restringir CORS.
- Base de datos: actualizar modelos.sql para incluir todas las tablas usadas por el código o ajustar el código para usar exactamente el modelo definido.
- Testing: reemplazar pruebas dummy por pruebas sobre parser, validadores 1FN/2FN/3FN, generador_sql, API y políticas de base de datos.
- Despliegue: validar que ambos servicios, Streamlit y FastAPI, queden levantados con puertos correctos y variables configuradas.

f) Estándares de datos y trazabilidad

Toda validación debe poder ser trazada al usuario, fecha, formato de entrada, esquema original, nivel detectado, violaciones, sugerencias, nivel objetivo, nivel final y esquema resultante. La estructura JSONB de historial_validaciones permite conservar evidencia técnica,



pero se recomienda limitar el tamaño de registros, validar contenido y aplicar limpieza de datos cuando corresponda.

CONCLUSIONES

- El Documento de Visión del proyecto DataQuest establece una línea base más completa para el desarrollo del sistema, vinculando el contenido del FD02 con la estructura real del ZIP revisado.
- DataQuest es un sistema técnicamente orientado a validar esquemas relacionales mediante Python, Streamlit, FastAPI, Supabase/PostgreSQL, Docker, Terraform y GitHub Actions, con un enfoque académico y funcional claro.
- El núcleo del producto se concentra en parsear SQL, detectar violaciones de 1FN, 2FN y 3FN, generar sugerencias, aplicar mejoras referenciales, visualizar diagramas y guardar historial de validaciones.
- El proyecto presenta una propuesta sólida, pero aún contiene pendientes importantes: parser CSV/XLS incompleto, funciones de clausura y claves candidatas sin implementar, pruebas dummy y discrepancia entre Supabase Auth documentado y autenticación manual implementada.
- La API FastAPI amplía el alcance del producto, permitiendo que otros sistemas consuman el motor de diagnóstico, aunque requiere controles adicionales de seguridad, CORS, tamaño de entrada y monitoreo para producción.
- El sistema puede considerarse viable como MVP académico si se declaran sus limitaciones. Para uso productivo, debe fortalecerse seguridad, autenticación, pruebas, precisión del motor de dependencias y consistencia del modelo de base de datos.

RECOMENDACIONES

1. Alinear de inmediato la autenticación: elegir entre Supabase Auth real o tabla usuarios propia. En ambos casos, actualizar db/modelos.sql, db/auth.py y las políticas RLS para que no existan contradicciones.
2. Reemplazar el hash SHA-256 básico y los tokens dummy por un mecanismo seguro antes de cualquier despliegue público.
3. Completar core/dependencias.py con cálculo de clausura y claves candidatas para reducir la dependencia de heurísticas y mejorar rigor académico.
4. Implementar lectura real de CSV/XLS o retirar temporalmente esa promesa del alcance funcional hasta que esté terminada.
5. Crear pruebas unitarias reales para parser, 1FN, 2FN, 3FN, corrector, generador SQL y API; eliminar pruebas dummy.
6. Corregir el workflow de GitHub Actions si no existe tests/bdd, o crear dicha carpeta con escenarios BDD válidos.
7. Limitar CORS, tamaño del SQL enviado a la API y frecuencia de peticiones para evitar abuso del endpoint público.
8. Agregar documentación de instalación paso a paso, script de migración completo, ejemplos de SQL y capturas del flujo principal para sustentación.
9. Mantener visible en el sistema que el SQL generado es referencial y debe ser revisado por el usuario antes de aplicarse en una base de datos real.



10. Actualizar el FD02 cuando el equipo complete las funcionalidades pendientes, para mantener trazabilidad entre documento, código y entregable final.

BIBLIOGRAFÍA

- Date, C. J. (2004). An Introduction to Database Systems. Pearson.
- Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems. Pearson.
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts. McGraw-Hill.
- Project Management Institute. (2017). A Guide to the Project Management Body of Knowledge (PMBOK® Guide) (6th ed.). Project Management Institute.

WEBGRAFÍA

- Documentación oficial de Python. <https://docs.python.org/>
- Documentación oficial de Streamlit. <https://docs.streamlit.io/>
- Documentación oficial de FastAPI. <https://fastapi.tiangolo.com/>
- Documentación oficial de Supabase. <https://supabase.com/docs>
- Documentación oficial de PostgreSQL. <https://www.postgresql.org/docs/>
- Documentación oficial de Docker. <https://docs.docker.com/>
- Documentación oficial de Terraform Azure Provider. <https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs>
- Documentación oficial de Mermaid. <https://mermaid.js.org/>